Ontario Tech University
Faculty of Science, Computer Science — CSCI 3010U

Student Number: 100785833 April 10, 2026

2D Billiards Physics Simulation Using Python

by

Yash Patel

Abstract: For my CSCI 3010U course project, I built a two-dimensional billiards simulation using Python and the Pygame library. The simulation places eight balls on a table and models how they move, bounce off walls, and collide with each other using real physics. Ball motion is updated with the Euler method, wall bouncing reverses the ball's velocity, and ball–ball collisions are resolved with Newton's Law of Restitution. I also added damping to simulate friction. Three experiments tested how the restitution and damping coefficients change the system's kinetic energy over time. The results matched what physics theory predicts, confirming that the simulation is working correctly. **Keywords:** Billiards, Euler method, collision detection, Newton's Law of Restitution, damping, kinetic energy

1 Introduction

Computer simulations let us study physical systems on a screen without needing the real thing. A billiards table is a perfect example: it has several balls that move, bounce, and collide, and the rules of physics are clear and testable. For this project, I decided to simulate a 2D billiards table because it directly applies the topics we covered in class: numerical integration, rigid-body collisions, and energy analysis. My goal was to build a simulation that:

- Updates each ball's position every frame using the Euler method.
- Bounces balls off the table walls correctly.
- Handles collisions between balls using momentum equations.
- Simulates friction so balls eventually slow down.
- Lets me run experiments to test how different parameters affect the system.

The final result is an interactive Pygame window where I can watch 8 balls bounce around in real time, and a separate `analysis.py` script that runs experiments and saves graphs of kinetic energy over time.

2 Physics and Implementation

Ball Properties and Motion (Euler Method)

Each ball is an object with a position (x, y), a velocity (v_x, v_y), a radius, and a mass. Every frame, I advance the ball's position using the Euler method:

$$x(t + \Delta t) = x(t) + v_x \cdot \Delta t \tag{1}$$

$$y(t + \Delta t) = y(t) + v_y \cdot \Delta t \tag{2}$$

where $\Delta t = 1/120\,\mathrm{s}$ is the time step. The Euler method is the simplest numerical integration approach — it is easy to implement and works well for small time steps. Friction is added by reducing the ball's speed each step:

$$v(t + \Delta t) = v(t) \cdot (1 - \lambda\,\Delta t) \tag{3}$$

where λ is the damping coefficient. A higher λ means the ball slows down faster.

Ball–Wall Collisions

When a ball hits a wall, I reverse the velocity component pointing into that wall:

- Left or right wall: $v_x \leftarrow -v_x$
- Top or bottom wall: $v_y \leftarrow -v_y$

I also nudge the ball back inside the boundary to prevent it from getting stuck inside the wall. Wall collisions are always elastic — no energy is lost.

Ball–Ball Collisions

Collision detection checks whether the distance between two ball centres is less than the sum of their radii. If a collision is found, I use Newton's Law of Restitution to calculate the impulse J:

$$J = \frac{-(1 + e) \cdot dv_n}{\dfrac{1}{m_1} + \dfrac{1}{m_2}} \tag{4}$$

where e is the restitution coefficient ($0 \leq e \leq 1$) and dv_n is the relative velocity along the collision normal. The velocities of both balls are then updated:

$$\mathbf{v}_1 \mathrel{+}= (J/m_1)\,\hat{n}, \qquad \mathbf{v}_2 \mathrel{-}= (J/m_2)\,\hat{n} \tag{5}$$

When $e = 1$ (perfectly elastic) no kinetic energy is lost. When $e < 1$ some energy is dissipated each collision.

Code Structure

The project has three Python files: `ball.py` defines the `Ball` class with the Euler update and wall-collision logic; `physics.py` contains the `resolve_ball_collision()` function; `main.py` runs the Pygame window with an interactive HUD showing live kinetic energy, collision count, and parameter values. Figure 1 shows a screenshot of the simulation.

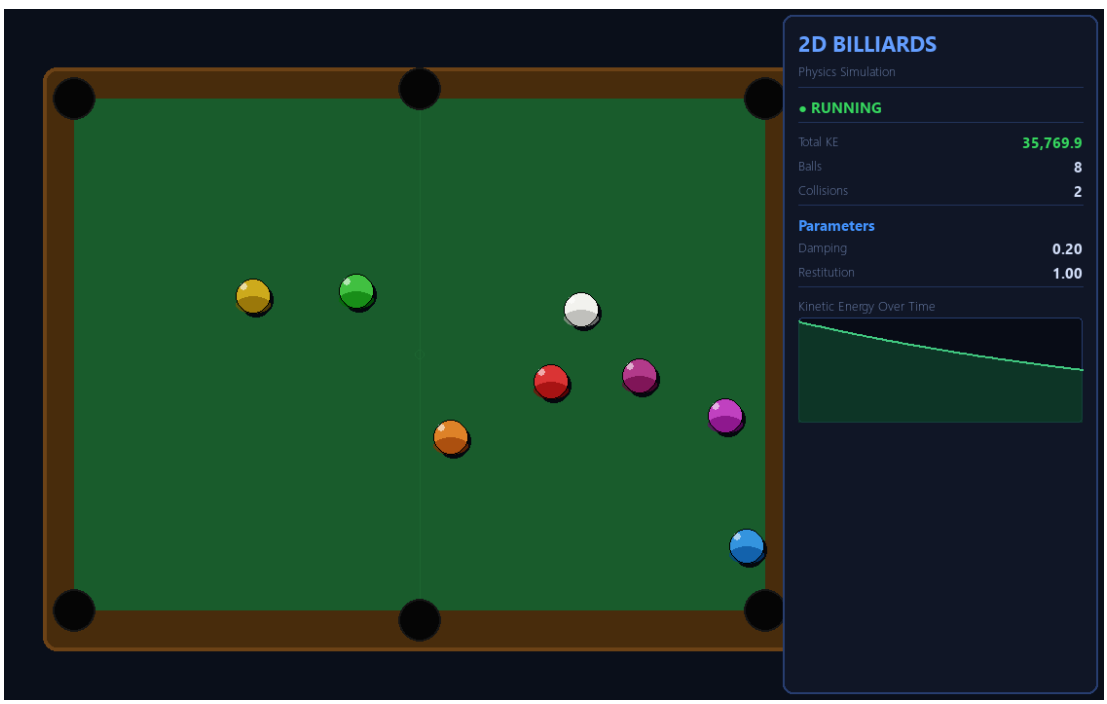

Figure 1: The interactive simulation window. Eight colour-coded balls move on a green felt table. The right panel shows real-time kinetic energy, collision count, and adjustable parameters.

3 Experiments and Results

I ran three experiments using `analysis.py`, which simulates 12 seconds of table time without a display window and saves kinetic energy data. All experiments start from the same initial ball positions (fixed random seed) to make results directly comparable. Figure 2 shows all three plots together. **Experiment 1 — Restitution.** I tested three restitution values ($e = 1.0,\ 0.7,\ 0.4$) with no damping. When $e = 1.0$ the kinetic energy stays perfectly flat — no energy is lost, which is exactly what an elastic collision means. For lower e values, the energy drops in steps: each step is one ball–ball collision that dissipates some energy. Lower e causes bigger drops per hit. **Experiment 2 — Damping.** I tested three damping values ($\lambda = 0.0,\ 0.2,\ 0.6$) with elastic collisions. Without friction the energy stays constant. With friction it decays smoothly and exponentially over time — at $\lambda = 0.6$ the balls stop within about four seconds. This matches the exponential decay that the damping equation predicts. **Experiment 3 — Momentum.** I tracked the magnitude of total system momentum over 12 seconds with elastic collisions and no damping. The momentum changes whenever a ball hits a wall, because the wall applies an external force. Ball–ball collisions do not change the total momentum, which is consistent with Newton's Third Law.

4 Conclusion

This project showed me how simulation techniques from class can be applied to a real physical system. I implemented the Euler method for ball motion, elastic wall reflections, impulse-based ball–ball collisions with a restitution coefficient, and a linear friction model. All three experiments produced results that matched theory: elastic collisions conserved energy, inelastic ones dissipated it, damping caused exponential decay, and momentum was conserved between ball–ball collisions. If I were to improve this project, I would try the Runge–Kutta (RK4) method instead of Euler for better numerical accuracy, and I would add proper pocket detection so balls can be "potted" like in real billiards. Overall, building this simulation helped me understand how physics equations translate into working code.

References

[1] H. Gould, J. Tobochnik, and W. Christian, *An Introduction to Computer Simulation Methods*, 3rd ed. Addison-Wesley, 2007.

[2] Pygame Community, *Pygame Documentation*, 2024. `https://www.pygame.org/docs/`

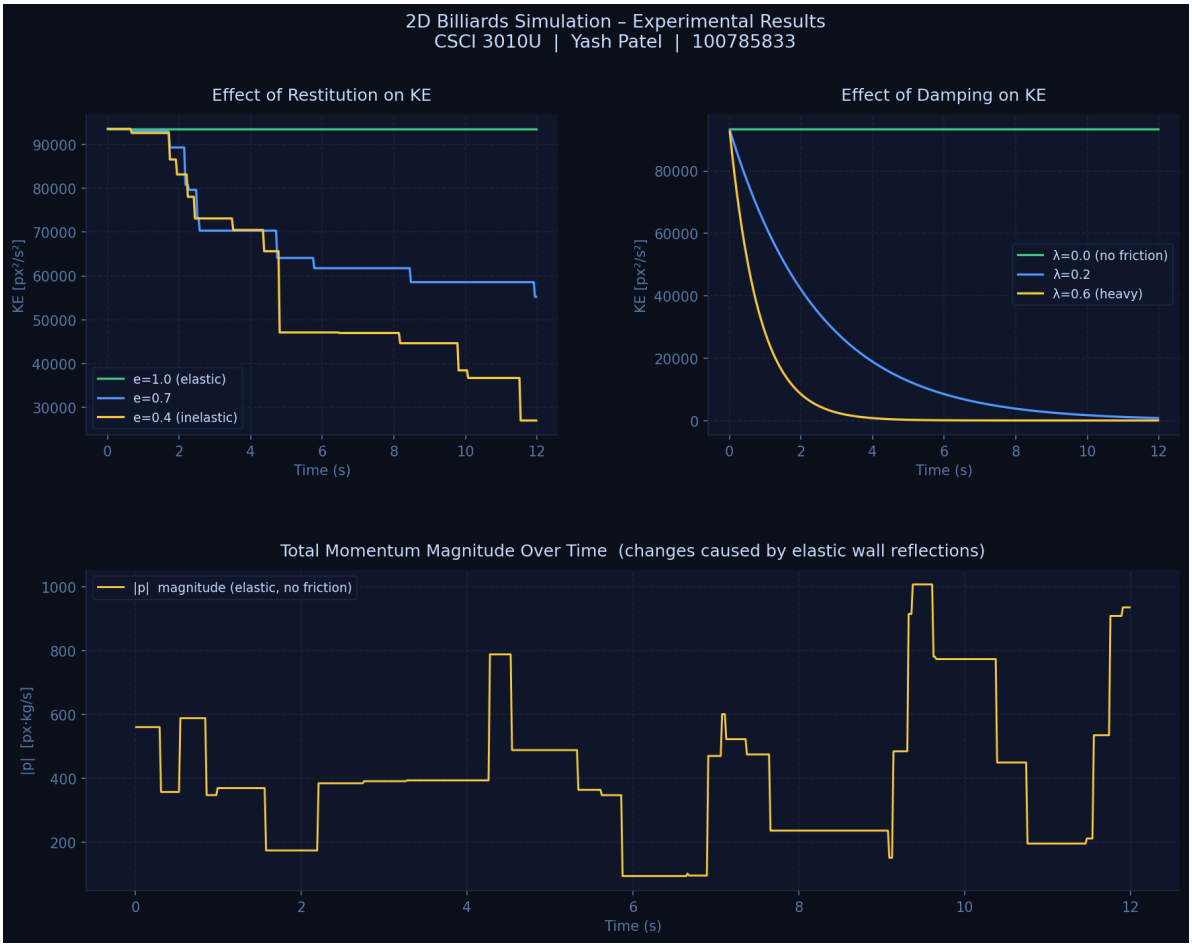

Figure 2: Experiment results. *Top-left:* kinetic energy vs. time for three restitution values. *Top-right:* kinetic energy vs. time for three damping values. *Bottom:* total momentum magnitude vs. time.